• Atividade 1 - teórica:

(1)

a) $5n^2 + n$ é $O(n^2)$. Para provar isso, preciso tomar m e c (constantes), tal que $c.f(n) \geq 5n^2 + n$, onde $n \geq m$.

tome $c = 6$.

Temos $6n^2 \geq 5n^2 + n$, $\forall n \geq 1$. $(m = 1)$.

Logo, $5n^2 + n = O(n^2)$.

b) $\underbrace{2n^4 + 2n^2}_{g(n)}$ é $\Omega(n^4)$ → Isso significa que o melhor caso é n^4.

quero provar que $g(n) \geq c.f(n)$, $\forall n \geq m$. Dados c e m (constantes).

temos,

$$2n^4 + 2n^2 \geqslant \underbrace{(1)}_{c} n^4, \quad \forall n \geqslant \underbrace{1}_{m}$$

Logo,

$2n^4 + 2n^2 = \Omega(n^4)$

c) Verificando as afirmações Ⓘ $2^{n+1} = O(2^n)$ e Ⓘ$\!$Ⓘ $2^{2n} = O(2^n)$

Ⓘ Podemos escrever $2^{n+1} = 2.2^n$, então

$$2^{n+1} \leqslant \underbrace{2}_{c}.\underbrace{2^n}_{f(n)}, \quad \forall n \geqslant \underbrace{0}_{m}$$

Isso é exatamente a definição de Big-O.

Portanto,

$2^{n+1} = O(2^n)$. verdadeiro.

Ⓘ$\!$Ⓘ Agora, temos $2^{2n} = (2^2)^n = 4^n$.

Isso significa que, quando tendemos ao infinito, $4^n = 2^{2n}$ cresce muito mais rapidamente que $2n$.

Portanto, $2^{2n} \neq O(2^n)$, ela seria na verdade $O(4^n)$. falso.

d) A ordem é:

$$1 < 4n < 2n \log n < (\log n!) < n2^n < 2^{2n} < (n+1)!$$

Analisando:

a. 1 (é constante)
b. $4n = O(n)$
c. $n \cdot \log n > n$, logo $4n < 2n \log n$. $2n \log n = O(n \log n)$
d. o fatorial de $\log n$ cresce mais rapidamente que $n \log n$.
   logo, $2n \log n < (\log n)!$
e. $n2^n$ cresce exponencialmente, enquanto $(\log n)!$ tem crescimento subexponencial em relação a n.
   logo, $(\log n)! < n2^n$.
f. $2^{2n} = (2^2)^n$ (já provamos na atividade anterior que isso é $O(4^n)$). logo, quando tendemos ao infinito, 4^n cresce mais rapidamente que $n2^n$.
g. o fatorial cresce mais rapidamente que exponenciais como 4^n.
   logo, $2^{2n} < (n+1)!$

2) A análise de complexidade é o estudo de quanto um algoritmo consome de recursos (processamento e memória (tempo de processamento e espaço).

O objetivo não é necessariamente descobrir o tempo exato que o programa levará em segundos, pois isso depende do computador, linguagem, compilador, etc. Mas sim, descobrir como o custo do algoritmo cresce conforme n aumenta.

Fazemos isso identificando, primeiro, o tamanho da entrada, representado por n. E depois analisamos quantas vezes as principais operações do código são executadas (loops e funções recursivas, por exemplo).

Aprendemos na primeira aula que bons programadores pensam em estratégias para otimizar seus algoritmos, e algumas "leis" essenciais para isso, são: evitar loops e funções recursivas, pensando em "arquiteturas" mais sofisticadas e complexas para os códigos.